Stockade

Dash-cam Collision-Risk Pipeline — Project Status Report

Summary of work completed on the detection/tracking and risk-scoring pipeline, the plan for what comes next, and the bottlenecks currently limiting progress.

Prepared: October 7, 2026

Project folder: Stockade (local, no version control yet)

Covers: Phase 1 (detection & tracking), Phase 2 (risk scoring v1–v3)

1. What We Did

Phase 1 — Detection & Tracking (`track_vehicles.py`)

COMPLETE — runs a pretrained YOLOv8 detector with ByteTrack over dash-cam clips, producing an annotated video (boxes + persistent track IDs) and a per-clip trajectory CSV for every vehicle detected. This is the data foundation everything else reads from.

- Verified working on sample clips — annotated outputs exist for multiple clips (e.g. `000577` , `000865`) with stable track IDs.
- Recursively finds all `.mp4` clips under an input folder and mirrors the folder structure in the output.
- Streams frame-by-frame rather than loading full clips into memory, so it can scale to large datasets without a rewrite.

Phase 2 — Risk Scoring (`score_risk.py`)

IN PROGRESS — a deliberately transparent, rule-based (no ML) scorer that turns each vehicle's frame-by-frame trajectory into a 0–1 risk score using two geometric signals: closing speed (box-area growth rate, proxy for time-to-collision) and lateral weaving (variance in horizontal position). This is the baseline that any future learned model has to beat.

This has already gone through three iterations:

Version	What changed	Result
v1	Single frame-to-frame diff for closing speed	Very low recall — most real crashes scored near zero
v2	Windowed regression slope instead of 2-point diff, box area instead of height, confidence filtering, proximity gate	52% recall but very high false-alarm rate; a logic bug silently zeroed the core signal
v3	Fixed the persistence-gate bug (it was mathematically impossible to trigger), dropped an inert "peer-relative" check, retuned thresholds for precision	See evaluation below — fewer false alarms, but recall dropped as a direct tradeoff

Latest Evaluation (v3, ~48 labeled DAD clips)

Threshold	Recall	Precision	False-alarm rate
0.3	52% (12/23 crashes caught)	52%	48%
0.5	39% (9/23 crashes caught)	60%	26%
0.7	13% (3/23 crashes caught)	60%	9%

No single threshold is good on both axes — this is the expected ceiling for a two-signal geometric rule, not a bug.

Data Prepared

- **CCD / CCD-Normal** (Car Crash Dataset) — extracted and usable.
- **DAD** (labeled crash/normal clips) — extracted, 1,750 clips total; only ~48 processed so far for evaluation.
- **nuScenes-mini** — extracted (camera/LIDAR/RADAR samples + metadata), not yet used by either script.
- V2X / cooperative-infrastructure datasets (~11GB) — left zipped, intentionally deferred to a later networked-version phase.

2. Next Plan

Short term — close out Phase 2 properly

- Run Phase 1 + Phase 2 across the **full** DAD set (1,750 clips), not just the 48-clip sample, to get a statistically meaningful read on recall/precision.
- Decide a working threshold (likely 0.5) and lock it in as the documented baseline, with the precision/recall tradeoff stated explicitly rather than tuned further by hand.
- Spot-check a batch of false positives and false negatives visually (via the annotated videos) to see if there's a cheap third signal worth adding before moving to ML — e.g. ego-motion compensation, since the comments already note whole-scene effects (ego braking) confound the current signals.

Medium term — move past the rule-based baseline

- Use the Phase 1 trajectory CSVs as training features for a learned model (e.g. gradient-boosted trees on hand-engineered features first, before anything heavier) — the rule-based v1–v3 scorer exists specifically to give this a baseline to beat.
- Bring in **nuScenes-mini** for multi-sensor validation (camera + RADAR/LIDAR ground truth) once the single-camera pipeline is stable, since it's extracted but unused.
- Re-run the same evaluation harness (`score_risk.py`'s metrics block) against the learned model so the comparison to the rule-based baseline is apples-to-apples.

Longer term — networked / V2X phase

- Unpack and incorporate the V2X-Seq and V2X-Sim datasets once the single-vehicle pipeline is validated at scale — this is explicitly out of scope until then.
- Extend risk scoring from a single dash-cam's view to a cooperative multi-vehicle view using that data.

3. Bottlenecks

Rule-based scorer has a hard accuracy ceiling. At the most balanced threshold (0.5), it still misses 61% of real crashes or, at a looser threshold, false-alarms on nearly half of normal clips. Two geometric signals (closing speed + weaving) can't resolve this on their own — this is the main thing blocking a usable Phase 2 outcome, not a tuning problem.

Evaluation set is tiny relative to the data available. Metrics are based on ~48 of 1,750 DAD clips. Current recall/precision numbers could shift meaningfully once run at full scale — the "60% precision" result isn't trustworthy yet as a final number.

No learned model yet. Everything built so far is intentionally a transparent, hand-tuned baseline. Progress toward an actually deployable risk model hasn't started — Phase 2's job was only to produce something to beat.

No version control. The project folder isn't a git repository, so there's no history of the v1→v2→v3 scoring changes beyond what's written in code comments, and no safety net if a future change regresses the scorer.

~15GB of datasets sitting unextracted/unused. The V2X/cooperative-infrastructure zips are fine to leave deferred, but they're consuming significant disk space for a phase that's not being worked on yet.